UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

Simularea funcționării unei memorii cache

Facultatea: Automatică și Calculatoare

Specializarea: Calculatoare și Tehnologia Informației

Disciplina: Structura sistemelor de calcul

***Student**: Puicar Georgiana-Oana*

***Grupa**: 30234*

Cuprins

1. Propunerea de proiect 4

1.1. Scopul proiectului 4

2. **Planificare 4**

2.1. Etapele proiectului 4

3. **Studiu bibliografic 4**

3.1. Principiile de funcționare 5

3.2. Organizarea memoriei cache 5

3.3. Tipuri de mapare 5

3.4. Politici de înlocuire și de scriere 5

3.5. Performanța cache-ului 6

3.6. Necesitatea simulării 6

4. **Analiză 7**

4.1. Scopul simulării 7

4.2. Operațiile principale ale memoriei cache 8

4.3. Politici de mapare și înlocuire implementate 8

4.4. Politici de scriere simulate 9

4.5. Parametri de performanță urmăriți 10

4.6. Funcționalitățile principale implementate 10

4.7. Testare și interpretare 10

5. **Design 11**

5.1. Scopul designului 11

5.2. Interfața grafică (GUI) 11

5.3. Funcționalitățile de bază 13

6. **Implementare 15**

6.1. Prezentare generală 15

6.2. Clase și componente 15

6.2.1. CacheLine 16

6.2.2. CacheCellRenderer 16

6.2.3. CacheMemory 17

6.2.4. CacheSimulatorGUI 17

6.2.5. StatsManager 18

7. **Testare..... 21**

7.1. Pregătirea mediului 21

7.2. Test 1 – Operații de bază 21

7.3. Test 2 – Politici de înlocuire (LRU, FIFO, Random) 21

7.4. Test 3 – Politici de scriere (Write-Through vs Write-Back) 22
7.5. Test 4 – Tipuri de mapare 22
7.6. Test 5 – Scenarii de performanță 22
7.7. Test 6 – Salvare log și statistici 23
7.8. Test 7 – Edge cases 23
7.9. Interpretarea rezultatelor 23

8. Referințe bibliografice 24

1. Propunerea de proiect

Scopul acestui proiect este crearea unei aplicații grafice interactive care să simuleze operarea memorii cache, care este o parte esențială a arhitecturii oricărui sistem de calcul contemporan.

Memoria cache se află între procesor și memoria principală și ajută la reducerea timpului de acces la date. Scopul principal al acestui proiect este de a vedea și de a analiza modul în care memoriei cache se comportă atunci când sunt efectuate diferite tipuri de operații, cum ar fi citirea și scrierea. De asemenea, se concentrează asupra modului în care politicile de mapare și înlocuire afectează performanța.

Scopul proiectului este de a oferi studenților o înțelegere a fundamentelor organizației memoriei cache și a modului în care diferitele reguli de funcționare afectează performanța sistemului în ansamblu.
Aplicația va fi implementată într-un limbaj de programare orientat pe obiecte (Java) astfel încât să poată fi utilizată ușor.

2. Planificare

▪ **Săptămâna 4 (22 oct):** propunerea proiectului, planificarea etapelor de lucru, studiu bibliografic
▪ **Săptămâna 6 (5 nov):** definirea structurilor de date și a funcțiilor pentru operațiile principale ale memoriei cache (plasare, regăsire, descărcare)
▪ **Săptămâna 8 (19 nov):** implementarea codului pentru simularea logicii interne a memoriei cache și testarea funcțiilor de bază
▪ **Săptămâna 10 (3 dec):** realizarea interfeței grafice și integrarea acesteia cu logica simulării
▪ **Săptămâna 12 (17 dec):** testarea finală, analiza rezultatelor și definitivarea documentației

3. Studiu bibliografic

Modul în care este organizată memoria unui sistem de calcul are un impact semnificativ asupra performanței acestuia. În timp ce procesoarele actuale execută instrucțiuni la viteze remarcabil de rapide, accesul la memoria principală (RAM) continuă să fie mult mai lent. Dacă procesorul ar fi nevoit să aștepte pentru fiecare operație de citire și scriere din RAM, această diferență de viteză ar provoca întârzieri semnificative.

Pentru a rezolva această problemă, arhitectura contemporană include un nivel intermediar de memorie, cunoscut sub numele de memorie cache, care păstrează local copii ale datelor și instrucțiunilor cele mai frecvent utilizate.

3.1 Principiile de funcționare

Principiul localității este baza memoriei cache:
• **Localitatea temporală**: un element de memorie este probabil să fie accesat din nou în viitorul apropiat dacă este accesat.
• **Localitatea spațială**: dacă o locație este accesabilă, este probabil ca și locațiile din apropiere să fie accesibile în curând.
Prin urmare, memoria cache stochează linii de date, cunoscute și sub numele de blocuri de date, fiecare fiind asociat cu o regiune contigue din memoria principală.

Astfel, memoria cache stochează blocuri (linii) de date, fiecare corespunzând unei regiuni contigue din memoria principală.

3.2 Organizarea memoriei cache

Memoria cache este organizată în

- Seturi: grupuri de linii
- Liniile sau blocurile: unitățile de stocare de bază
- Etichetele (tag-uri): identificatori distincți care conectează o linie din cache cu blocul corespunzător din memoria principală

Împărțirea adresei în trei părți este necesară pentru adresarea unei locații din cache:
• **Eticheta:** ajută la identificarea blocului de memorie;
• **Index**: selectează setul pentru a fi salvat în cache;
• **Offset:** îți indică poziția exactă în bloc.

3.3 Tipuri de mapare

Există trei scheme principale de mapare între adresele memoriei principale și liniile cache:

1. **Mapare directã (Direct-Mapped Cache)** – fiecare bloc din memoria principală are o singură poziție posibilă în cache. Este simplă, dar predispusă la coliziuni.

2. **Mapare complet asociativã (Fully Associative Cache)** – un bloc poate fi plasat oriunde în cache, eliminând coliziunile, dar necesitând mai multe comparatoare hardware.

3. **Mapare set-asociativã (N-Way Set Associative)** – compromis între primele două, în care cache-ul este împărțit în seturi de câte *N* linii, iar un bloc poate fi plasat în oricare dintre liniile setului.

3.4 Politici de înlocuire și de scriere

Atunci când cache-ul este plin, o linie trebuie înlocuită. Printre cele mai utilizate politici se numără:

- **LRU (Least Recently Used)** – elimină blocul care nu a mai fost folosit de cel mai mult timp;
- **FIFO (First In First Out)** – elimină blocul introdus cel mai vechi;
- **Random** – elimină un bloc ales aleator.

Pentru operațiile de scriere, se folosesc:

- **Write-through** – datele sunt scrise simultan și în cache, și în memoria principală;
- **Write-back** – datele sunt scrise doar în cache, iar în memoria principală doar la înlocuire (dacă linia este „dirty”);
- **Write-allocate** – la o scriere pe un miss, blocul este adus în cache înainte de actualizare.

3.5 Performanța cache-ului

Performanța unei memorii cache este determinată de:

- **rata de hit** – procentul de accese găsite în cache;
- **rata de miss** – procentul de accese care au necesitat citirea din memoria principală;
- **timpul mediu de acces (Average Memory Access Time – AMAT)**, calculat ca:

$$\text{AMAT} = T_{hit} + (\text{Miss Rate}) \times T_{miss}$$

unde T_{hit} este timpul de acces în cache, iar T_{miss} este timpul de acces la memoria principală.

Un cache bine configurat reduce considerabil timpul total de execuție al programelor, prin minimizarea numărului de accese la memoria lentă.

3.6 Necesitatea simulării

Simularea funcționării memoriei cache este utilă atât pentru studenți, cât și pentru proiectanții de arhitecturi hardware. Un simulator software oferă:

- o modalitate vizuală de **înțelegere a comportamentului cache-ului**;
- posibilitatea de **comparare a politicilor** de înlocuire și scriere;
- analiza **efectului dimensiunii și asociativității** asupra performanței.

Astfel, proiectul propus oferă un instrument didactic valoros pentru studierea principiilor de proiectare a ierarhiei de memorie.

4. Analiză

Programul își propune să simuleze în mod interactiv funcționarea unei memorii cache, oferind utilizatorului posibilitatea de a vedea procesele interne care au loc în timpul proceselor de citire, scriere și înlocuire a datelor. Aplicația va sublinia diferențele dintre principalele politici de mapare și înlocuire și modul în care acestea afectează performanța sistemului la scară largă.

4.1. Scopul simulării

Simularea își propune să demonstreze modul în care funcționează memoria cache prin următoarele operații: plasarea unui bloc de date nou; găsirea datelor care au fost deja încărcate prin hituri sau erori de cache; și înlocuirea unui bloc atunci când memoria cache este plină prin descărcare sau înlocuire.
Utilizatorul va fi capabil să înțeleagă concepte precum asociativitatea cache-ului (direct-mapped, set-associative, fully associative), politicile de înlocuire (FIFO, LRU, Random), politicile de scriere (Write-through, Write-back) și comportamentul sistemului în caz de hit și miss cache.

4.2. Operațiile principale ale memoriei cache

Operațiile principale ale memoriei cache sunt următoarele:

a) Plasarea unei noi informații

Acest lucru se întâmplă atunci când procesorul solicită un bloc de date care nu se află în cache, sau atunci când nu este în cache. Blocul specific va fi încărcat din memoria principală a aplicației și apoi plasat într-o linie de cache liberă.
Dacă memoria cache este plină, politica de înlocuire preferată (de exemplu FIFO sau LRU) va fi utilizată pentru a determina ce bloc va fi eliminat.

b) Regăsirea unei informații

O lovitură în cache, sau o lovitură în cache, apare atunci când procesorul solicită un bloc de date care este deja în cache. În acest scenariu, timpul de acces este redus, iar programul va marca grafic linia de cache relevantă cu o culoare diferită.
În cazul unui cache miss, linia solicitată este încărcată din memoria principală și este afișată ca o operație mai lentă. Acest lucru este indicat vizual printr-o marcă roșie.

c) Descărcarea unei informații

Când o linie de cache trebuie înlocuită din cauza lipsei de spațiu, se face o descărcare.
Înainte de a fi înlocuite, datele vor fi scrise în memoria principală înainte de a fi înlocuite dacă politica de scriere este Write-back și linia este marcată ca dirty.

Un log detaliat în interfața grafică permite vizualizarea acestei operațiuni.

4.3. Politici de mapare și înlocuire implementate

Programul va permite alegerea modului de mapare:

- ***Direct-mapped***

Definiție:
Fiecare bloc de memorie are o singură poziție posibilă în cache.

Implementare:
Indexul este derivat direct din adresă folosind operația modulo; dacă poziția este ocupată, blocul existent este înlocuit pe baza politicii de scriere.

index = address % cacheSize

tag = address / cacheSize

- ***Set-associative***

Definiție:
Cache-ul este împărțit în seturi, iar un bloc poate fi plasat în orice linie din setul corespunzător.

Implementare:
Adresa determină setul; în interiorul setului se aplică politica de înlocuire (FIFO, LRU, Random).

numSets = cacheSize / ways

setIndex = (address % numSets) × ways

tag = address / numSets

- ***Fully-associative***

Definiție:
Un bloc poate fi plasat în orice linie a cache-ului.

Implementare:
Nu există index; se caută tagul în toate liniile, iar la nevoie se aplică politica de înlocuire globală.

De asemenea, utilizatorul va putea selecta o **politică de înlocuire**, cum ar fi:

FIFO (First In, First Out)

Definiție:
Politica FIFO înlocuiește întotdeauna blocul care se află de cel mai mult timp în cache, adică blocul încărcat cel mai devreme. Este similară cu o coadă: primul intrat este primul scos.

Implementare:

- Fiecare linie păstrează un câmp insertionOrder care se actualizează la inserare
- La înlocuire, se selectează linia cu valoarea insertionOrder cea mai mică
- Funcționează ca o coadă: primul intrat este primul scos

LRU (Least Recently Used)

Definiție:
LRU elimină blocul care **nu a mai fost folosit de cel mai mult timp**. Politica ține cont de istoricul accesărilor reale, nu doar de ordinea inserării.

Implementare:
Fiecare linie are un atribut **lastUsed** (timp logic), care se actualizează:

- la fiecare hit -> lastUsed devine un număr mai mare (de ex. +1 față de globalCounter);
- la fiecare miss -> când aleg blocul pentru înlocuire, caut linia cu lastUsed *cel mai mic*.

Random

Definiție:
Politica Random alege **aleator** o linie din set atunci când trebuie făcută înlocuirea. Nu necesită urmărirea timpilor sau ordinii accesării. Folosește un generator de numere aleatoare pentru a selecta o linie din intervalul disponibil

Dacă există linii libere (invalid), acestea sunt preferate

4.4. Politici de scriere simulate

Programul va simula și diferența dintre:

- ***Write-through***

Definiție:
Orice scriere în cache este imediat propagată și în memoria principală.

Implementare:
- La operația de write, cache-ul și memoria principală sunt actualizate simultan;

- Liniile nu devin „dirty" deoarece RAM-ul este mereu sincronizat;

- Asigură consistența datelor, dar poate fi mai lentă;

- ***Write-back***

Definiție:
Datele sunt scrise în memoria principală doar atunci când linia modificată este înlocuită.

Implementare:
Scrierea marchează linia ca „dirty”; la înlocuire, dacă linia este dirty, datele sunt salvate în memoria principală.

4.5. Parametri de performanță urmăriți

Simulatorul va calcula și afișa:

- **Rata de cache hit** – procentul de accesări care au găsit datele în cache
- **Rata de cache miss** – procentul de accesări care au necesitat citirea din memoria principală
- **Numărul total de operații efectuate** (citiri + scrieri)
- **Timpul mediu de acces estimat** – bazat pe valori predefinite:
    - Timp acces cache: 1.0 ns
    - Timp acces RAM: 100.0 ns
    - Formula: AMAT = 1.0 + (missRate × 100.0)

Aceste date sunt afișate atât în interfața grafică (label de statistici), cât și în fișierul de statistici salvat.

4.6. Funcționalitățile principale implementate

Pentru organizarea codului, aplicația va conține mai multe clase și funcții modulare, fiecare având un rol clar:

- **CacheLine** – stochează informațiile despre o linie (tag, data, valid, dirty, lastUsed, insertionOrder)
- **CacheMemory** – gestionează logica de plasare, regăsire și înlocuire
- **CacheCellRenderer** – componenta grafică pentru evidențierea vizuală a operațiilor
- **StatsManager** – calculează și gestionează parametrii de performanță
- **CacheSimulatorGUI** – interfața grafică Swing care coordonează întreaga aplicație

4.7. Testare și interpretare:

Aplicația va fi testată în mai multe configurații diferite:
• dimensiuni de cache diferite (4, 8, 16 linii);

• politici de înlocuire diferite (FIFO, LRU);

• serii de acces la memorie secvențial și aleator;
Oferind utilizatorului o înțelegere vizuală și numerică a comportamentului memoriei cache, rezultatele vor fi interpretate prin comparație între ratele de hit/miss și timpul mediu de acces.

5. Design

5.1. Scopul designului

Programul își propune să simuleze vizual funcționarea unei **memorii cache**, oferind utilizatorului o interfață grafică intuitivă pentru observarea pașilor de citire, scriere și actualizare a datelor.
Designul urmărește:

- **Claritate** – informații prezentate clar și structurat
- **Interactivitate** – răspuns imediat la acțiunile utilizatorului
- **Ușurință în utilizare** – controale simple și intuitive
- **Feedback vizual** – culori și mesaje descriptive pentru fiecare operație
-

Principalele obiective:

- Vizualizarea în timp real a operațiilor cache (hit, miss, write-back)
- Configurare dinamică a parametrilor cache
- Feedback detaliat și descriptiv pentru fiecare operație
- Menținerea unei structuri modulare în cod

5.2. Interfața grafică (GUI)

Aplicația va fi realizată în **Java**, folosind biblioteca **Swing** pentru partea grafică.
Fereastra principală va fi împărțită în trei zone:

1. **Zona de control (input și comenzi)**

Primul rând:

- **Câmp text** pentru introducerea adresei de memorie (0-49)
- **Buton Citire** – pentru operații de read
- **Buton Scriere** – pentru operații de write
- **Buton Reset** – pentru golirea cache-ului

- **Buton Salvare Stats** – pentru exportul statisticilor

Al doilea rând (configurare):

- **Dimensiune cache**: 4, 8 sau 16 linii
- **Mapare**: Direct, Set-Associative, Fully Associative
- **Înlocuire**: LRU, FIFO, Random
- **Scriere**: Write-Back, Write-Through

Toate combo box-urile resetează cache-ul la modificare pentru a evita stările inconsistente.

2. Zona grafică a memoriei cache (centru)

Tabel JTable cu 5 coloane:

- **Index** – numărul liniei (0, 1, 2, ...)
- **Valid** – ✓ (validă) sau ✗ (invalidă)
- **Tag** – eticheta blocului de memorie
- **Dirty** – D (modificată) sau - (curată)
- **Date** – conținutul liniei sau "(gol)"

Colorare dinamică:

- **Verde deschis** – Cache hit (date găsite)
- **Roz deschis** – Cache miss (date încărcate din RAM)
- **Alb** – Linii neafectate

Culoarea se actualizează instantaneu după fiecare operație folosind CacheCellRenderer.

3. Zona de statistici și log (partea de jos)

- Label de statistici (verde, bold):
- Hits: X | Misses: Y | Total: Z | Hit Rate: W% | Miss Rate: V% | Avg Time: T ns
- Zonă de log (text area cu scroll):

- Mesaje descriptive pentru fiecare operație
- Informații despre hit/miss, linii înlocuite, politici aplicate
- Salvare automată în fișierul cache_log.txt

5.3. Funcționalitățile de bază

1. Operația de Citire (Read)

Flux de execuție:

1. Utilizatorul introduce o adresă (0-49) și apasă **Citire**
2. Sistemul verifică dacă adresa există în cache (checkHit)
3. **Dacă HIT**:
    - Linia este marcată verde
    - Se afișează mesaj: "✓ CACHE HIT la linia X"
    - Statisticile sunt actualizate
4. **Dacă MISS**:
    - Datele sunt încărcate din RAM
    - Se selectează o linie pentru înlocuire (conform politicii)
    - Dacă linia înlocuită e dirty (Write-Back), se scrie în RAM
    - Linia este marcată roz
    - Se afișează: "✗ CACHE MISS - Date încărcate din RAM"

2. Operația de Scriere (Write)

Flux de execuție:

1. Utilizatorul introduce o adresă și apasă **Scriere**
2. Se generează date noi: "Date_X_modificate"
3. Sistemul verifică dacă adresa există în cache
4. **Write-Back**:
    - Dacă HIT: se actualizează cache-ul, dirty = true
    - Dacă MISS: se încarcă în cache, dirty = true

- RAM-ul nu e actualizat imediat

5. **Write-Through**:
    - Cache-ul și RAM-ul sunt actualizate simultan
    - dirty = false (mereu sincronizat)

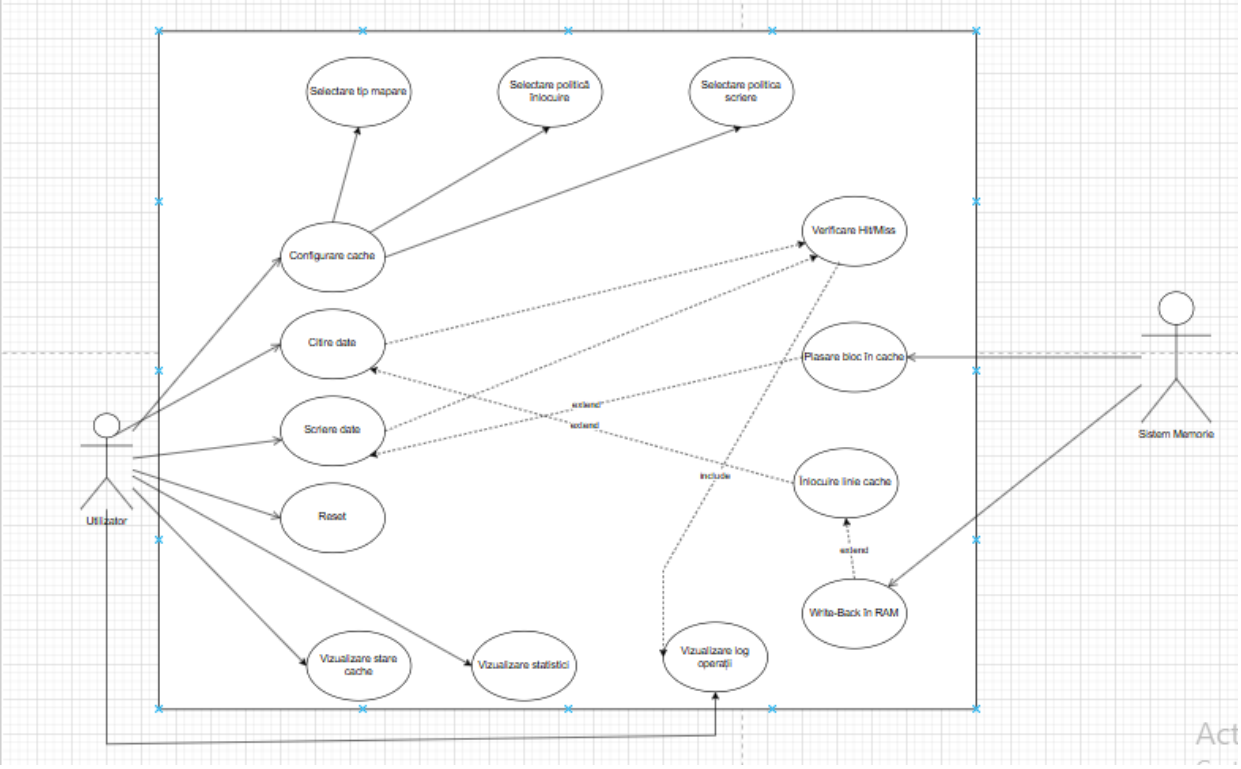
3. Operația de Reset

- Golește toate liniile cache (valid = false)
- Resetează statisticile (hits = 0, misses = 0)
- Șterge evidențierea vizuală
- Păstrează configurația curentă (dimensiune, politici)

4. Salvare Statistici

- Exportă un raport detaliat în cache_statistics.txt
- Include: timestamp, configurație, număr operații, performanță, timing
- Mesaj de confirmare în interfață

Diagrama Use Case:



6. Implementare

Această secțiune descrie implementarea practică a proiectului — un simulator de memorie cache cu interfață grafică. Porțiunea de mai jos explică arhitectura componentelor software, interfeţele (atribute/metode), comportamentul principal, fluxurile de date şi semnale (evenimente), precum şi elementele auxiliare pentru testare şi vizualizare.

6.1. Prezentare generală

Simulatorul oferă o implementare didactică a unui cache de memorie ce suportă:

- tipuri de mapare: **Direct**, **Set-Associative**, **Fully Associative**;
- politici de înlocuire: **LRU**, **FIFO**, **RANDOM**;
- politici de scriere: **Write-Back**, **Write-Through**;
- interfață grafică pentru citire/scriere, configurare și vizualizare (tabel, log, statistici).
- export de date: log-uri și statistici în fișiere text

Proiectul este împărțit pe clase Java, fiecare având responsabilități clar delimitate:

- CacheLine — model pentru o linie de cache (stare, tag, date, dirty, timestamp).
- CacheMemory — logică & politici cache (mapare, înlocuire, load/place/get).
- CacheCellRenderer — componentă pentru evidenţierea rândului afectat în GUI.
- CacheSimulatorGUI — interfaţa grafică (Swing) și orchestratorul operațiilor.
- StatsManager — gestionează parametrii de performanță.

6.3. Clase și componente

Următoarea descriere detaliază fiecare componentă, interfețe și fluxuri de execuţie.

1. **CacheLine — linia de cache**

Descriere:
Clasa CacheLine reprezintă o singură linie din memoria cache, păstrând informațiile esențiale pentru gestionarea datelor: eticheta (tag), datele efective, validitatea, starea *dirty* și indicatori pentru politicile de înlocuire LRU și FIFO. Practic, fiecare obiect CacheLine simulează o „celulă” a cache-ului care poate fi accesată sau înlocuită.

Funcționalitate principală:

- Reține dacă linia conține date valide.
- Păstrează informații pentru determinarea liniei de înlocuit în funcție de politica cache-ului.
- Servește ca structură de bază pentru CacheMemory.

```java
package org.example;

public class CacheLine {  8 usages
    public int tag;  9 usages
    public String data;
    public boolean valid;  14 usages
    public boolean dirty;  4 usages
    public long lastUsed;  6 usages
    public long insertionOrder;  4 usages

    public CacheLine() {  1 usage
        this.valid = false;
        this.dirty = false;
        this.tag = -1;
        this.data = "";
        this.lastUsed = 0;
        this.insertionOrder = 0;
    }
}
```

2. **CacheCellRenderer — evidențierea liniilor în GUI**

Descriere:
Clasa CacheCellRenderer personaliza afișarea tabelului (JTable) care reprezintă memoria cache în GUI. Rolul său principal este să evidențieze vizual linia cache afectată de ultima operațiune: *hit* sau *miss*. Aceasta ajută utilizatorul să urmărească în timp real comportamentul cache-ului.

Funcționalitate principală:

- Păstrează informația despre ultima linie accesată (lastOperationLine) și tipul operațiunii (lastOperationType).
- Modifică culoarea de fundal a rândului corespunzător în funcție de rezultat:
- Verde deschis -> cache *hit*

- Roz deschis -> cache *miss*
- Alinierea textului în celule este centrată.

```java
@Override
public Component getTableCellRendererComponent(
        JTable table, Object value, boolean isSelected,
        boolean hasFocus, int row, int column) {

    Component c = super.getTableCellRendererComponent(
            table, value, isSelected, hasFocus, row, column);

    if (row == lastOperationLine) {
        if ("hit".equals(lastOperationType)) {
            c.setBackground(new Color( r: 144,  g: 238,  b: 144));
        } else if ("miss".equals(lastOperationType)) {
            c.setBackground(new Color( r: 255,  g: 182,  b: 193));
        }
    } else {
        c.setBackground(Color.WHITE);
    }

    setHorizontalAlignment(SwingConstants.CENTER);
    return c;
}
```

3. **CacheMemory — motorul cache**

Descriere:
Clasa CacheMemory simulează comportamentul unei memorii cache. Ea gestionează liniile cache, deciziile de înlocuire, maparea adreselor și politica de scriere. Este responsabilă pentru determinarea dacă o adresă este prezentă în cache (*hit*) sau nu (*miss*), alegerea liniei de înlocuit, plasarea datelor și sincronizarea cu memoria principală simulată.

- **Funcționalitate principală:**
- Stochează un array de CacheLine care reprezintă liniile cache.
- Suportă diferite politici:

- **ReplacementPolicy:** LRU, FIFO, RANDOM
- **MappingType:** Direct, Set-associative, Fully-associative
- **WritePolicy:** Write-back, Write-through

- Determină indexul și tag-ul asociat unei adrese.
- Verifică dacă un acces este *hit* sau *miss*.
- Alege linia de cache de înlocuit conform politicii curente.
- Plasează sau actualizează o linie în cache, scriind în memoria principală dacă e cazul.

```
public boolean isHit(int address) { 1 usage
    if(mappingType == MappingType.FULLY_ASSOCIATIVE) {
        for(int i=0;i<cacheSize;i++) {
            if(cache[i].valid && cache[i].tag == getTag(address)) {
                cache[i].lastUsed = counter++;
                return true;
            }
        }
        return false;
    } else if(mappingType == MappingType.SET_ASSOCIATIVE) {
        int start = getIndex(address);
        for(int i=start;i<start+ways;i++) {
            if(cache[i].valid && cache[i].tag == getTag(address)) {
                cache[i].lastUsed = counter++;
                return true;
            }
        }
        return false;
    } else {
        int index = getIndex(address);
        return cache[index].valid && cache[index].tag == getTag(address);
    }
}
```

4. **CacheSimulatorGUI — Interfața grafică a simulatorului de cache**

Descriere:
Clasa CacheSimulatorGUI reprezintă interfața grafică (GUI) a simulatorului de memorie cache. Aceasta permite utilizatorului să interacționeze cu memoria cache simulată prin citiri și scrieri, să configureze

parametrii cache-ului (dimensiune, tip mapare, politică de înlocuire, politică de scriere) și să vizualizeze starea cache-ului, statistici și log-ul operațiunilor.
Clasa gestionează și memoria principală simulată (mainMemory) și realizează sincronizarea datelor între aceasta și cache în funcție de politica de scriere selectată.

Funcționalitate principală:

- Creează și afișează tabelul cu liniile cache și evidențiază ultima operațiune (hit/miss).
- Permite citirea (readOperation) și scrierea (writeOperation) datelor în cache și RAM simulată.
- Actualizează log-ul operațiunilor și statistici (hits, misses, hit rate).
- Suportă resetarea cache-ului și reconfigurarea dinamică a dimensiunii și politicilor.

```
private void updateCacheDisplay() {  5 usages
    tableModel.setRowCount(0);
    CacheLine[] lines = cache.getCache();

    for (int i = 0; i < lines.length; i++) {
        CacheLine line = lines[i];
        tableModel.addRow(new Object[]{
                i,
                line.valid ? "✓" : "×",
                line.valid ? line.tag : "-",
                line.dirty ? " D" : "-",
                line.valid ? line.data : "(gol)"
        });
    }

    cacheTable.repaint();
}
```

5. StatsManager — Gestionarea statisticilor cache-ului
Descriere:
Clasa **StatsManager** se ocupă de monitorizarea și calcularea statisticilor legate de funcționarea memoriei cache. Înregistrează hit-uri, miss-uri, operații de citire și scriere și calculează indicatori precum *hit rate*, *miss rate* și *timpul mediu de acces*. De asemenea, poate genera rapoarte detaliate despre performanță și configurația cache-ului, pe care le poate salva într-un fișier text.

Funcționalitate principală:

- Înregistrează *hits*, *misses*, citiri și scrieri.
- Calculează *hit rate*, *miss rate* și *timpul mediu de acces*.
- Oferă statistici formatate, simple sau detaliate.
- Permite resetarea statisticilor la reconfigurarea cache-ului.
- Poate salva statisticile într-un fișier extern pentru analiză.

```
public boolean saveStats(String mappingType, String replacementPolicy, String writePolicy) {
    try (PrintWriter writer = new PrintWriter(new FileWriter(statsFilePath, append: true))) {
        writer.print(getDetailedStats(mappingType, replacementPolicy, writePolicy));
        return true;
    } catch (IOException e) {
        System.err.println("Eroare la salvarea statisticilor: " + e.getMessage());
        return false;
    }
}
```

7. Testare

Această secțiune prezintă un ghid scurt pentru testarea simulatorului de memorie cache. Scopul este verificarea corectitudinii funcționării și înțelegerea modului în care diferitele politici influențează comportamentul cache-ului.

7.1. Pregătirea mediului

Necesită:

- Un IDE Java sau linia de comandă
- Proiectul compilat

Pași:

1. Deschide proiectul în IDE.
2. Rulați clasa **CacheSimulatorGUI**.
3. Verificați că interfața afișează zona de Control, Tabelul Cache și Log + Statistici.
4. La pornire, tabelul trebuie să aibă 8 linii invalide, iar statistica să arate 0 hit-uri/miss-uri.

7.2. Test 1 – Operații de bază

Configurație: Direct Mapping, LRU, Write-Back, 8 linii.

Ce se testează:

- Primul acces la o adresă -> *cache miss*
- Acces repetat -> *cache hit*
- Citiri pe adrese diferite -> completarea cache-ului
- Scriere în modul Write-Back -> linia devine *dirty*
- Înlocuire LRU -> o linie este scoasă în funcție de ultima utilizare
- Write-Back la înlocuire -> datele dirty sunt scrise în RAM

Acest test confirmă funcționarea de bază a citirilor/scrierilor, evidențierea hit/miss, actualizarea tabelului și principiile politicii Write-Back.

7.3. Test 2 – Politici de înlocuire (LRU, FIFO, Random)

Scop: Observarea diferențelor dintre politici.

Idee generală:

- **FIFO** înlocuiește primul bloc adăugat.
- **LRU** înlocuiește blocul cel mai puțin folosit recent.
- **Random** alege la întâmplare.

Exemplu:
Umpleți cache-ul cu 8 adrese, apoi accesați o adresă care generează coliziune.

- Cu FIFO -> prima linie introdusă este înlocuită.
- Cu LRU -> se înlocuiește linia nefolosită recent (nu neapărat prima).

7.4. Test 3 – Politici de scriere (Write-Through vs Write-Back)

Write-Through:

- Scrierea actualizează simultan cache-ul și RAM.
- Liniile *nu devin dirty*.

Write-Back:

- Scrierea actualizează doar cache-ul -> linia devine *dirty*.
- RAM-ul se actualizează doar la înlocuire.

Test: scrieți pe o adresă și verificați dacă RAM-ul se actualizează imediat (Write-Through) sau doar la înlocuire (Write-Back).

7.5. Test 4 – Tipuri de mapare

Direct Mapping:

- Fiecare adresă are o singură linie posibilă.
- Coliziuni frecvente.

Fully Associative:

- O adresă poate ajunge în orice linie.
- Cele mai puține coliziuni.

Set-Associative:

- Compromis între primele două.
- Înlocuirile se fac doar în interiorul setului.

Scenariile de citire a unor adrese precum 0, 8, 16 ilustrează clar diferențele dintre aceste moduri.

7.6. Test 5 – Scenarii de performanță

Acces secvențial:

- Prima trecere -> multe miss-uri
- A doua trecere -> multe hit-uri
- Demonstrează localitatea spațială.

Acces aleator:

- Hit rate scăzut
- Demonstrează impactul lipsei unui pattern.

Cache mai mic:

- Mai multe coliziuni și înlocuiri
- Performanță mai slabă.

7.7. Test 6 – Salvare log și statistici

1. Efectuați un set de operații.
2. Apăsați **„Salvare Stats”**.
3. Verificați fișierele:
    - **cache_log.txt** -> toate operațiile efectuate
    - **cache_statistics.txt** -> rapoarte detaliate pentru fiecare salvare

Fișierul de statistici se actualizează în mod *append*, deci toate rapoartele rămân salvate.

7.8. Test 7 – Edge cases

- Introducerea unor adrese invalide -> mesaj de eroare
- Cache plin -> se testează corectitudinea înlocuirilor
- Reset în timpul utilizării -> liniile devin invalide, statisticile se resetează, log-ul rămâne intact

7.9. Interpretarea rezultatelor

- **Hit Rate mare (>80%)** -> acces cu localitate bună
- **Hit Rate mic (<30%)** -> acces aleator sau cache insuficient
- **Average Access Time** scăzut -> puține miss-uri
- **Politici**:
    - LRU = cea mai eficientă în general
    - FIFO = simplă, dar suboptimală
    - Random = rar utilizată
- **Politici de scriere**:
    - Write-Through -> acuratețe, overhead mare
    - Write-Back -> performanță bună, implementare mai complexă

8. Referințe bibliografice

1. David A. Patterson, John L. Hennessy – *Computer Organization and Design: The Hardware/Software Interface*, 6th Edition, Morgan Kaufmann, 2020.
2. Wikipedia – Cache (computing), Cache replacement policies, Write-through, Write-back. Disponibil la: https://en.wikipedia.org/wiki/Cache_(computing)
3. Cursuri de *Arhitectura Calculatoarelor*, Universitatea Politehnica București, Facultatea de Automatică și Calculatoare, 2024.
4. Oracle Java Documentation – Swing Tutorial. Disponibil la: https://docs.oracle.com/javase/tutorial/uiswing/